

T.C Bursa Teknik Üniversitesi
Mühendislik ve Doğa Bilimleri Fakültesi Bilgisayar Mühendisliği Bölümü
BLM0238 Programlama Dilleri Dönem Projesi
GO Programlama Dili Analizi
Hamza ÖZÜTÜRK 25360859334

1. Giriş

Go programlama dili (Golang), 2007 yılında Google'da Robert Griesemer, Rob Pike ve Ken Thompson tarafından geliştirilmeye başlanmıştır. 2009 yılında açık kaynak olarak duyurulmuştur. Dilin kökeni, doğrudan C dilinin verimliliğine ve Pascal/Oberon ailesinin tip güvenliğine ve yapısal sadeliğine dayanıyor.

Go'nun temel çıkış amacı aslında günümüzdeki o karmaşık bilgi işlem dünyasını basitleştirmektir.

Günümüzde çekirdekli sistemler ve devasa kod tabanları kullanımda. çok çekirdekli işlemciler, ağ sistemleri ve büyük kod tabanlarıyla doludur. C++ ve Java gibi dillerin aşırı kalıtım hiyerarşileri, uzun derleme süreleri ve karmaşık bağımlılık yönetimleri, Go'nun tasarımında tamamen reddedilmiştir. GO sistem programlama yeteneklerini korurken derleme hızını optimize etmiş ve geliştirici üretkenliğini ön plana çıkarmaktadır.

2. DERLEYİCİ VE SÖZDİZİMİ

Derste işlenen **Halting Problem** ve derleyici tasarım mimarileri bağlamında Go, doğrudan makine koduna çevrilen (compiled) statik bir dildir. Java'daki gibi aracı bir sanal makine (JVM) kullanmaz. Ayırıştırma (Parsing) aşamasında, dilin yalnızca 25 anahtar kelimeden oluşan minimalist yapısı sayesinde **LL(1)** mantığıyla çok hızlı Abstract Syntax Tree (AST) oluşturulur. Sözcüksel analizör (Lexer), C'deki gibi noktalı virgül zorunluluğunu kaldırarak satır sonlarına örtük olarak noktalı virgül ekler.

3. VERİ TIPLERİ VE TİP GÜVENLİĞİ

Dil tasarımında en kritik güvenlik önlemlerinden biri tip dönüşümüdür. C ve C++ dillerinde esneklik adına **Coercion (Örtük/Otomatik Tip Dönüşümü)** yapılırken, Go bunu tamamen reddeder. Bir int32 ile int64 toplanamaz; derleyici **Type Error** verir. Tip eşdeğerliğinde ise struct'lar için **Name Equivalence** (isim bazlı katı kontrol), arayüzler için ise **Structure Equivalence** (yapısal eşdeğerlilik) kullanır

```
// Coercion yasaktır, Explicit (Açık) dönüşüm şarttır
var x int32 = 10

var y float64 = 20.5

// sonuc := x + y // Derleme hatası!
```

4. Scope ve Binding

Go, tamamen Statik (Lexical) Kapsam kullanır. Derste Dinamik Kapsam için anlatılan Deep Access (aktivasyon kayıtlarında geriye dönük arama) veya Shallow Access mekanizmaları Go'da kesinlikle bulunmaz. Python gibi dillerde kapsam çözünürlüğü çalışma zamanında esneklik gösterirken, C ve Java gibi dillerde olduğu gibi Go'da da bir değişkenin bellek offset adresi derleme zamanında (compile-time) kesin olarak tespit edilir. Bu durum, çağrı zinciri (call stack) ne kadar derin olursa olsun çalışma zamanında değişkenlere O(1) sürede erişim sağlar. İsim-nesne bağlaması (binding), kısa atama operatörü := ile statik tip çıkarımı (type inference) yapılarak o anki yerel environment'a bağlanır.

5. İFADELER VE KONTROL

C dillerinde atama işlemi bir ifade (expression) olarak değer döndürürken (örn: a = b = c), Go'da atama bir deyimdir (statement) ve değer döndürmez. Kontrol akışında while veya do-while yoktur; pretest ve posttest tüm döngüler for ile sağlanır. Ayrıca çoklu seçim yapılarında (switch) derste gördüğümüz **Fall-through (Aşağı kayma)** tehlikesi Go'da varsayılan olarak kapalıdır; blok sonunda otomatik break devreye girer.

6. ALT PROGRAMLAR VE BELLEK ETKİLEŞİMİ

Alt programlar (fonksiyonlar) çağrıldığında yığılma (Stack) bir Aktivasyon Kaydı (Activation Record) oluşur. Parametre geçiş yöntemi olarak Go her zaman Pass-by-Value (Değer ile Geçiş) kullanır.

Referans geçişi (Pass-by-reference) yoktur; adresi değiştirmek istiyorsak işaretçilerin (pointer) değerini kopyalayarak yollarız. Fonksiyonlar "Birinci Sınıf Vatandaş"tır. İç içe (nested) fonksiyonlar üst kapsamdaki bir değişkeni kullanırsa bir Closure oluşur ve bellek yönetimi bunu Stack'ten Heap'e kaydırır.

```
// Closure: sayaç heap'e taşınır
func sayici() func() int {

    i := 0

    return func() int { i++; return i }
```

7. SOYUTLAMA (ADT) VE JENERİK YAPILAR

Veri soyutlaması için class yerine struct kullanılır. Kapsülleme (Information Hiding) için public/private gibi anahtar kelimeler yerine isimlendirme kuralları geçerlidir; büyük harfle başlayan isimler dışa aktarılır (Exported), küçük harfle başlayanlar gizlenir. Go 1.18 ile birlikte derste işlenen C++ Template'lerine benzer Parametrik Polimorfizm (Generics) dile eklenmiştir.

```
// Jenerik (Generic) Stack ADT
type Stack[T any] struct {

    elemanlar []T
}

func (s *Stack[T]) Push(deger T) {
    s.elemanlar = append(s.elemanlar, deger)
```

8. Nesneye Yönelik Programlama(OOP)

Go'nun en farklılaştığı noktalardan biri, derin Sınıf Hiyerarşilerini (Class Hierarchy) ve Subclass Initialization (üst sınıfı super() ile başlatma) sorunlarını engellemek için Kalıtımı (Inheritance) tamamen yasaklamasıdır. Bunun yerine Kompozisyon (Composition) kullanılır; struct'lar birbirinin içine gömülür (Embedding). Dinamik bağlama (Dynamic Binding) ise Örtük Arayüzler (Implicit Interfaces) ile yapılır.

Özellikle Java ve C++ gibi geleneksel OOP dillerinde derin sınıf hiyerarşileri kurulduğunda, bir nesnenin davranışı tahmin edilemez hale gelebilir ve "Spaghetti Code" sorunu ortaya çıkar. Go'nun Örtük Arayüz (Implicit Interface) yapısı "Duck Typing" mantığına benzer ancak çalışma zamanında değil, derleme zamanında tip güvenliğini garanti eder. C veya Java'daki gibi arayüzler arası sıkı bağ (tight coupling) oluşturmadığı için, kodun yeniden kullanılabilirliğini maksimuma çıkarır.

```
// Kalıtım yerine Gömme (Embedding)
type Motor struct { guc int }

func (m Motor) Calis() {}

type Araba struct {

    Motor // Subclass yok, doğrudan göme
    renk string
```

9. EŞZAMANLILIK (CONCURRENCY)

Derste işlenen Flynn sınıflandırmasındaki MIMD mimarisine uygun olarak, Go işletim sistemi seviyesindeki Thread'ler yerine Goroutine adı verilen hafif sıkletli (lightweight) coroutineler kullanır. Paylaşılan bellek kilitlenmelerini (Deadlock) önlemek için Semaphore veya Monitör yerine CSP (Communicating Sequential Processes) modelini benimser. Goroutine'ler arası veri aktarımı Kanallar (Channels) aracılığıyla Mesaj Geçişi (Message Passing) yöntemiyle sağlanır.

```
func isci(kanal chan int) { kanal <- 100 }
func main() {

    ch := make(chan int)

    go isci(ch) // Goroutine başlatıldı
    deger := <-ch // Kanaldan veri okundu
```

10. İSTİSNA(EXCEPTIONS) VE HATALAR

Geleneksel try-catch-finally blokları programın kontrol akışını bozduğu için Go'da bulunmaz. Hatalar (errors) tıpkı normal değişkenler gibi birinci sınıf değerlerdir ve fonksiyonların son dönüş değeri olarak açıkça kontrol edilmek zorundadır. Beklenmedik fatal durumlarda ise panic, recover ve defer mekanizmaları ile programın temiz bir şekilde kapatılması veya kurtarılması sağlanır.

11. BELLEK YÖNETİMİ

C/C++'daki gibi manuel malloc / free yönetimi bellek sızıntılarına yol açar. Go, belleği eşzamanlı çalışan (concurrent) bir **Çöp Toplayıcı (Garbage Collector)** ile yönetir. Derleyici, **Kaçış Analizi (Escape Analysis)** yaparak yerel değişkenlerin Stack'te mi yoksa Heap'te mi tutulacağına karar verir. Bu durum programcının bellek yönetimi yükünü sıfıra indirirken performansın korunmasını sağlar.

Manuel bellek yönetimine sahip C dilinde programcılar malloc ve free ile yığın/öbek (stack/heap) kontrolünü tamamen kendi yaparken, pointer aritmetiği yüzünden ciddi güvenlik açıkları oluşur. Go ise C'nin bu donanım seviyesindeki hızını alırken, pointer aritmetiğine izin vermeyerek güvenliği sağlar. Kaçış analizi (Escape Analysis) mekanizması, bir fonksiyonun içindeki değişkenin fonksiyon dışına referans verip vermediğini (örneğin bir closure yaratıp yaratmadığını) tarar. Eğer referans dışarı taşıyorsa değişkeni Heap'e, taşımiyorsa Stack'e koyarak Çöp Toplayıcının (Garbage Collector) iş yükünü inanılmaz derecede hafifletir.

12. PARADİGMA DESTEĞİ

Go, çok paradigmalı bir dildir. Ağırlıklı olarak **Emirli (Imperative)** ve **Eşzamanlı (Concurrent)** bir tasarım sunar. Saf fonksiyonel diller (Haskell) gibi mutlak yan etkisizlik (immutability) dayatmaz, ancak birinci sınıf fonksiyonlar ve closure desteği ile fonksiyonel yaklaşımlara izin verir. Mantıksal dillerdeki (Prolog) gibi kural tabanlı bir çıkarım mekanizması yoktur, tamamen deterministik bir yürütme sergiler.

13. BU DİLİ DİĞERLERİNDEN AYIRAN EN ÖNEMLİ ÖZELLİĞİ

Go'yu diğer dillerden ayıran en temel 2 özellik

1. Çekirdeğe Gömülü Eşzamanlılık (Concurrency)

- Hafif İş Parçacıkları: Ağır işletim sistemi thread'leri yerine, bellek maliyeti yok denecek kadar az olan Goroutine'leri kullanır.
- Kilit Yerine Mesajlaşma: İş parçacıkları arasında veriyi paylaşırken kilit (Mutex/Semaphore) kullanmak yerine, veriyi Kanallar (Channels) üzerinden birbirlerine mesaj olarak atarlar (CSP Modeli). Bu sayede kilitlenme (deadlock) riskleri biter.

2. Sınıfsız ve Kalıtımsız OOP (Nesne Yönelimi)

- Kalıtım Yok: Go'da class ve extends (kalıtım) yoktur. Bunun yerine yapılar birbirinin içine gömülerek Kompozisyon kullanılır.
- Örtük Arayüzler (Implicit Interfaces): Bir nesnenin bir arayüzü sağlaması için implements yazmasına gerek yoktur; metotlara sahip olması yeterlidir. Bu da kodu inanılmaz esnek yapar.

14. MODERN MİMARİLERDE KULLANIMI

Derste görülen tüm bu dil paradigmaları (CSP tabanlı eşzamanlılık, kalıtımsız yapı, hızlı derleyici), Go'yu özellikle günümüz bulut bilişim yüksek performanslı backend sistemleri ve mikro-servis mimarilerinde endüstri standardı haline getirmiştir. Java'nın hantal JVM başlatma sürelerine veya Python'un yorumlayıcı (interpreter) yavaşlığına maruz kalmayan Go; kendi başına çalışabilen düşük bellek tüketen, çoklu çekirdekleri donanım seviyesinde verimli kullanan modüller üretir. Bu özellikler, özellikle anında derlenip ayağa kalkması gereken projelerde Go'yu diğer tüm alternatiflerin önüne geçirmektedir

15. Sonuç

Sonuç olarak Go programlama dili; C dilinin donanıma yakın performansını alırken, onun pointer aritmetiği ve manuel bellek yönetimi gibi tehlikeli yanlarını akıllı kaçış analizi (escape analysis) ve çöp toplayıcı (garbage collector) ile törpülemiştir.

Nesneye yönelik programlama tarafında işleri derin ve karmaşık sınıf hiyerarşileriyle zorlaştırmak yerine, sadece arayüzler ve kompozisyon kullanarak çok daha sade bir yapı sunuyor. Eşzamanlılık (concurrency) tarafında ise hepimizin korkulu rüyası olan kilitlenme (deadlock) sorunlarını klasik yöntemler yerine CSP kanallarıyla oldukça temiz bir şekilde çözmüş.

Kısacası Go; statik kapsamlı, tip güvenliğine sıkı sıkıya bağlı ve derleyici mimarisi oldukça minimalist olan, günümüz yazılım dünyasında hem güvenli hem de işimizi inanılmaz kolaylaştıran bir programlama dilidir.

Kaynakça

<https://go.dev/ref/spec>

[https://en.wikipedia.org/wiki/Go_\(programming_language\)](https://en.wikipedia.org/wiki/Go_(programming_language))

<https://go.dev/blog/intro-generics>

<https://go.dev/ref/mem>

Video link:

<https://youtu.be/zwdRI-PPNpE>